

Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

LABORATORY WORK 3:
EMPIRICAL ANALYSIS OF GRAPH TRAVERSAL
ALGORITHMS
(DEPTH FIRST SEARCH AND BREADTH FIRST SEARCH)

Author:

st. gr. FAF-242

Rusnac Nichita

Verified:

asist. univ.

Fistic Cristofor

Chisinau – 2026

Contents

ALGORITHM ANALYSIS	2
Objective	2
Tasks	2
Theoretical Notes	2
Introduction	3
Comparison Metric	3
Input Format	3
IMPLEMENTATION	5
Depth First Search (DFS)	5
Breadth First Search (BFS)	7
Graph Type Results	10
CONCLUSION	36
REFERENCES	37

ALGORITHM ANALYSIS

Objective

Study and empirically analyse two fundamental graph traversal algorithms: Depth First Search (DFS) and Breadth First Search (BFS), evaluating their execution time across different graph types and input sizes.

Tasks:

1. Implement DFS (recursive and iterative variants) and BFS in Python;
2. Establish the properties of the input data against which the analysis is performed;
3. Choose metrics for comparing the algorithms;
4. Perform empirical analysis of the proposed algorithms;
5. Make a graphical presentation of the obtained data;
6. Draw conclusions on the work done.

Theoretical Notes:

Empirical analysis complements theoretical complexity analysis by measuring actual program behaviour on a real machine. It is particularly valuable when:

- the theoretical bounds differ only by constant factors;
- the algorithm's performance depends heavily on the structure of the input (e.g. graph density, connectivity, diameter);
- comparing two algorithms with the same asymptotic complexity to find which is faster in practice.

The general empirical methodology followed in this work is:

1. Define the analysis goal and select the efficiency metric.
2. Identify the input properties: graph type, number of nodes n , edge density.
3. Implement the algorithms.
4. Generate multiple representative input data sets.
5. Execute the program for each data set and record the measurement.
6. Analyse and visualise the collected data.

Introduction:

Graph traversal is one of the most fundamental operations in computer science. DFS and BFS are two complementary strategies that visit every vertex of a graph exactly once, differing only in the order in which vertices are explored.

Both algorithms have the same asymptotic time complexity of $O(V + E)$, where V is the number of vertices and E is the number of edges. Despite this theoretical equivalence, their practical execution times differ because of the data structures they rely on (stack vs. queue), their memory access patterns, and how they interact with the structure of the graph.

This laboratory work investigates these practical differences by measuring wall-clock execution time across thirteen graph classes, covering sparse, dense, structured, directed, multi-edge, weighted, and regular topologies. The benchmarking is done through a C++ implementation exposed to Python with pybind11.

Comparison Metric:

The primary metric is **execution time measured in seconds** (wall-clock time, averaged over three independent repetitions to reduce measurement noise).

DFS and BFS both run in $O(V + E)$ time, so the quantity of interest is the *constant factor* difference introduced by the different internal data structures and traversal orders.

Input Format:

Graphs are represented as adjacency lists in a C++ backend exposed to Python. For visualization, small graphs with $n = 12$ are generated in Python. For benchmarks, the C++ module generates graphs with $n \in \{100, 200, 500, 1000, 2000\}$ under two density regimes (*sparse* and *dense*). Thirteen graph classes are used:

1. **Simple graph** – undirected graph without self-loops or parallel edges.
2. **Multigraph** – undirected graph allowing parallel edges.
3. **Pseudograph** – multigraph that also allows self-loops.
4. **Directed graph** – directed edges (arcs) without parallel edges.
5. **Directed multigraph** – directed edges with parallel arcs.
6. **Complete graph** – K_n , every pair of nodes connected.
7. **Wheels** – hub node connected to a cycle, W_n .
8. **Bipartite graph** – two partitions with edges only across partitions.

9. **Regular graph** – fixed degree graph (here 3-regular).
10. **Planar graph (grid-based)** – 2-D grid layout graph.
11. **Trees** – connected acyclic graphs.
12. **Cyclic graph** – cycle C_n .
13. **Weighted graph** – undirected graph with integer edge weights (weights ignored by BFS/DFS).

IMPLEMENTATION

The traversal algorithms and graph generators are implemented in C++ and exposed to Python through a pybind11 extension (`graph_algos.cpp`). The notebook `traversal_algorithms copy.ipynb` orchestrates the workflow:

1. build the C++ module (`setup.py build_ext --inplace`);
2. configure graph types, sizes, densities, and trial counts;
3. run benchmarks and collect timings;
4. generate traversal visualizations and result plots.

Depth First Search (DFS):

DFS explores a graph by going as deep as possible along each branch before backtracking. It uses a LIFO stack – either an explicit list (iterative variant) or Python’s call stack (recursive variant).

Theoretical Complexity:

- **Time:** $O(V + E)$ – each vertex is pushed and popped from the stack once; each edge is examined once (undirected: twice, once per endpoint).
- **Space:** $O(V)$ – the stack holds at most V elements (depth of the deepest reachable path from the source).

Classical DFS (C++ implementation):

Listing 1: Classical DFS with an explicit stack

```
1 static int classical_dfs(const std::vector<std::vector<int>> &adj, int
   source) {
2     int n = static_cast<int>(adj.size());
3     std::vector<char> visited(n, 0);
4     std::vector<int> stack;
5     stack.reserve(n);
6     stack.push_back(source);
7     int count = 0;
8
9     while (!stack.empty()) {
10         int u = stack.back();
11         stack.pop_back();
12         if (visited[u]) {
13             continue;
14         }
15         visited[u] = 1;
```

```

16     ++count;
17     const auto &nbrs = adj[u];
18     for (auto it = nbrs.rbegin(); it != nbrs.rend(); ++it) {
19         int v = *it;
20         if (!visited[v]) {
21             stack.push_back(v);
22         }
23     }
24 }
25 return count;
26 }

```

Optimized DFS (C++ implementation):

Listing 2: Optimized DFS using CSR storage

```

1 static int optimized_dfs_csr(const CSRMatrix &csr, int source) {
2     int n = csr.n;
3     static thread_local std::vector<char> visited;
4     static thread_local std::vector<int> stack;
5     visited.assign(n, 0);
6     stack.clear();
7     stack.reserve(n);
8
9     visited[source] = 1;
10    stack.push_back(source);
11    int count = 0;
12
13    while (!stack.empty()) {
14        int u = stack.back();
15        stack.pop_back();
16        ++count;
17        for (int idx = csr.row[u]; idx < csr.row[u + 1]; ++idx) {
18            int v = csr.col[idx];
19            if (!visited[v]) {
20                visited[v] = 1;
21                stack.push_back(v);
22            }
23        }
24    }
25    return count;
26 }

```

Results:

On sparse graphs, DFS execution time grows close to linearly with n , consistent with the $O(V + E)$ bound. In dense settings, adjacency scanning dominates runtime and both DFS and BFS scale more steeply. The optimized DFS uses CSR storage and reuses

buffers to reduce memory overhead and improve cache locality.

Optimizations:

- **CSR storage** keeps adjacency lists contiguous in memory, improving cache locality during neighbor scans.
- **Degree-based neighbor ordering** explores higher-degree neighbors first to improve cache behavior in dense regions.
- **Buffer reuse** uses thread-local arrays to avoid repeated allocations during benchmark loops.
- **Visited on push** avoids redundant stack entries and removes a visited check on pop.

Breadth First Search (BFS):

BFS explores a graph level by level using a FIFO queue. It guarantees that the first time a node is visited it has been reached via the *shortest path* (minimum number of hops) from the source node.

Theoretical Complexity:

- **Time:** $O(V + E)$ – each vertex is enqueued and dequeued exactly once; each edge is examined once per endpoint.
- **Space:** $O(V)$ – the queue may hold an entire BFS frontier layer, which in the worst case (e.g. star or complete graph) contains $O(V)$ nodes simultaneously.

Classical BFS (C++ implementation):

Listing 3: Classical BFS with a deque

```
1 static int classical_bfs(const std::vector<std::vector<int>> &adj, int
   source) {
2     int n = static_cast<int>(adj.size());
3     std::vector<char> visited(n, 0);
4     std::deque<int> q;
5     q.push_back(source);
6     visited[source] = 1;
7     int count = 1;
8
9     while (!q.empty()) {
10         int u = q.front();
11         q.pop_front();
12         for (int v : adj[u]) {
13             if (!visited[v]) {
14                 visited[v] = 1;
```



```

15         ++count;
16         q.push_back(v);
17     }
18 }
19 }
20 return count;
21 }

```

Optimized BFS (CSR + frontier array):

Listing 4: Optimized BFS using CSR storage

```

1 static int optimized_bfs_csr(const CSRMatrix &csr, int source) {
2     int n = csr.n;
3     std::vector<char> visited(n, 0);
4     std::vector<char> x(n, 0);
5     std::vector<char> y(n, 0);
6     std::vector<int> L(n, 0);
7
8     x[source] = 1;
9     visited[source] = 1;
10    L[0] = source;
11    int start = 0;
12    int end = 1;
13    int z = 1;
14    int count = 1;
15
16    while (start < end) {
17        for (int qi = start; qi < end; ++qi) {
18            int j = L[qi];
19            for (int idx = csr.row[j]; idx < csr.row[j + 1]; ++idx) {
20                int i = csr.col[idx];
21                if (!visited[i]) {
22                    y[i] = x[j] | y[i];
23                    visited[i] = 1;
24                    L[z] = i;
25                    ++z;
26                    ++count;
27                }
28            }
29            visited[j] = 1;
30            x[j] = 0;
31        }
32        start = end;
33        end = z;
34        x.swap(y);
35        std::fill(y.begin(), y.end(), 0);
36    }

```

```
37     return count;
38 }
```

Results:

BFS execution time also grows linearly in n for sparse regimes, while dense regimes rise more sharply due to larger adjacency scans. The optimized BFS uses CSR storage and a frontier array to reduce queue overhead.

Optimizations:

- **CSR storage** keeps adjacency lists contiguous in memory, improving cache locality during neighbor scans compared to vector-of-vectors.
- **Frontier array** (the L list and x/y flags) processes BFS levels in contiguous slices, which reduces queue overhead.
- **Buffer reuse** stores BFS arrays in thread-local buffers to reduce allocation costs across trials.
- **Visited on discovery** prevents the same node from being added to the frontier multiple times.

Graph Type Results:

Each graph type is reported with a short description, a traversal visualization for a small instance ($n = 12$), and benchmark results for sparse and dense variants (mean of three runs). The plots compare classical BFS/DFS versus optimized BFS/DFS (CSR-based optimizations and degree-ordered neighbors for DFS).

1. Simple Graph

Undirected graph without self-loops or parallel edges. This is the baseline topology used in the C++ generator, where edges are sampled uniformly at random subject to the target density. It provides the reference shape for comparing how the traversal cost changes with edge count.

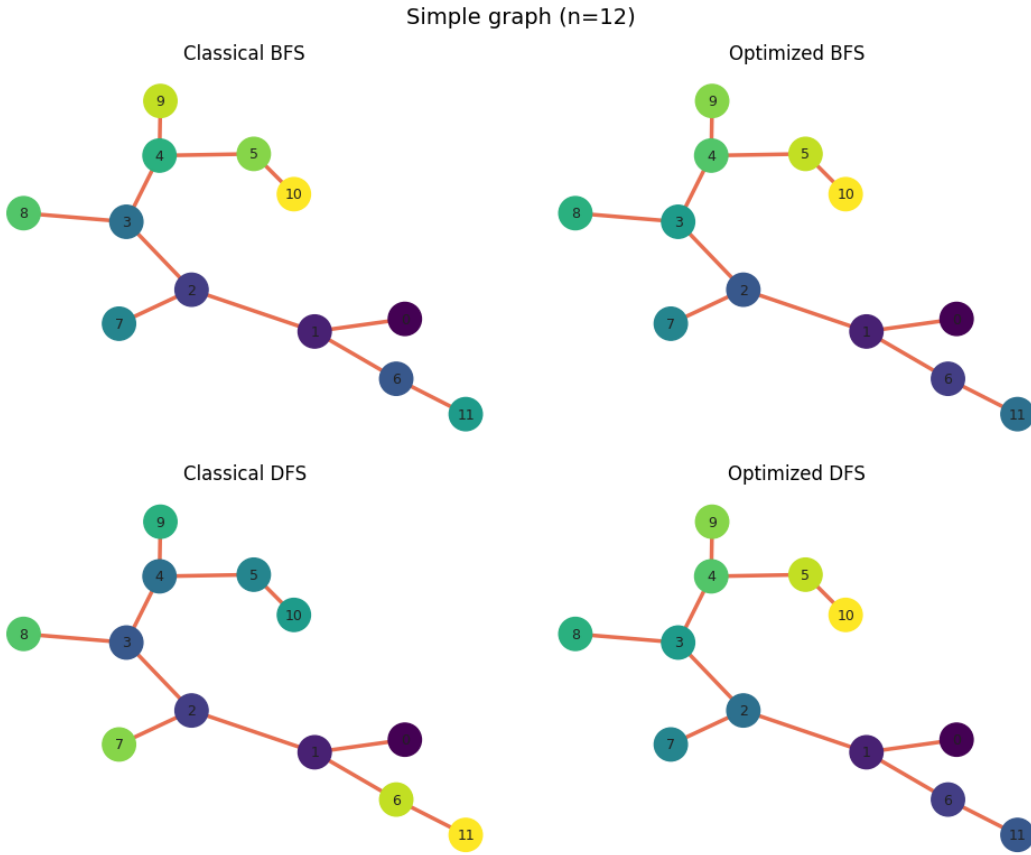


Figure 1: *Simple Graph Traversal* ($n=12$).

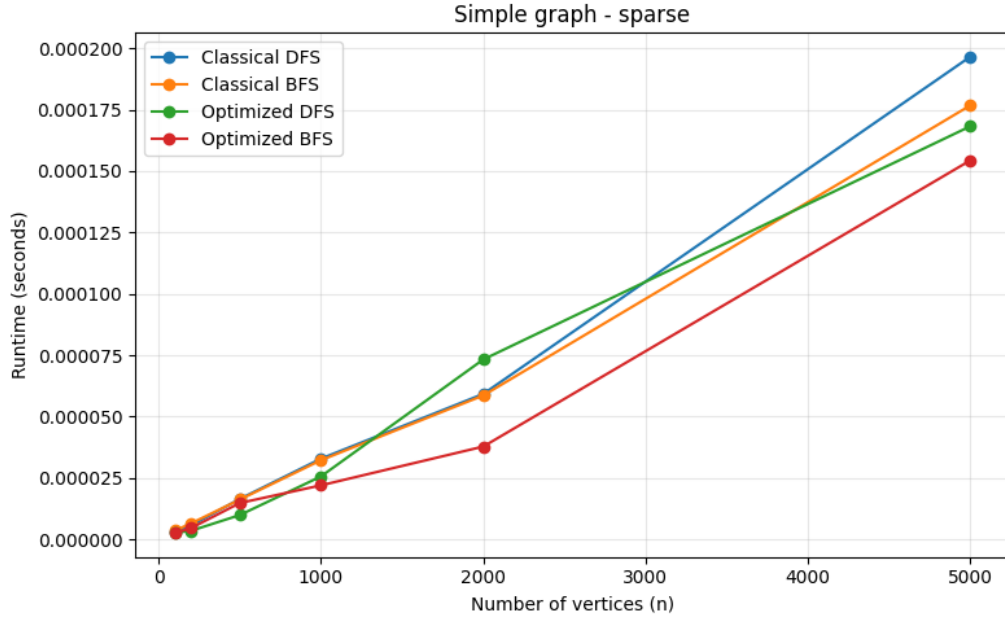


Figure 2: *Simple Graph Results – Sparse.*

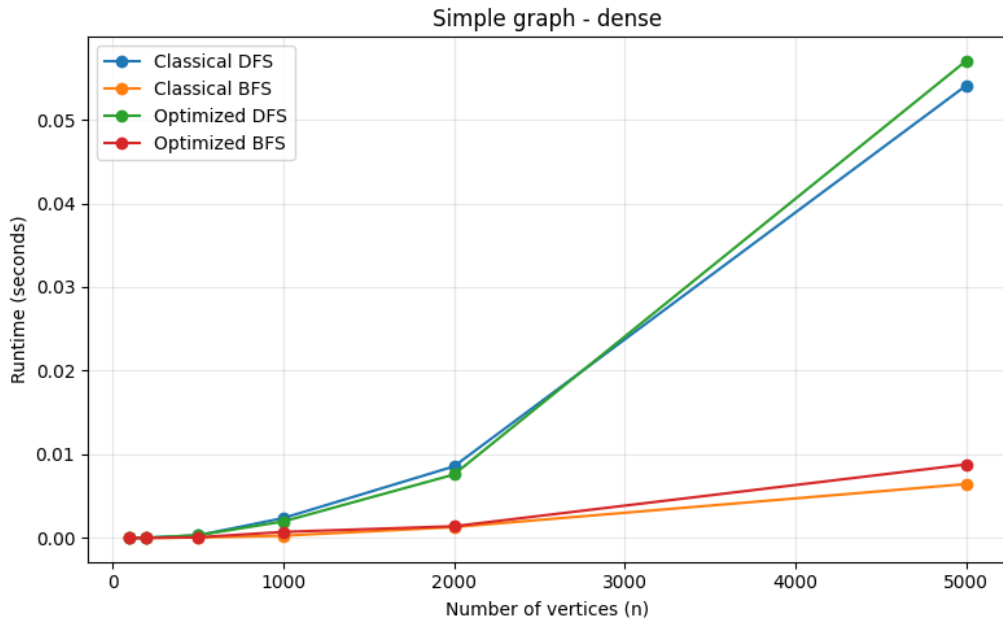


Figure 3: *Simple Graph Results – Dense.*

In sparse form, all four curves scale almost linearly with a small constant gap because E grows proportionally to V . In the dense case, the edge count grows quickly, so adjacency scans dominate and the curves rise much faster. The optimized BFS (CSR) can slightly reduce constant factors from better locality, while optimized DFS is identical to classical DFS, so the DFS curves nearly overlap.

2. Multigraph

Undirected graph with parallel edges in the visualization, but the benchmark generator deduplicates edges into a simple adjacency list. As a result, the timed version behaves like a simple random graph with the same density target. This makes multigraph results a close proxy for the simple-graph results.

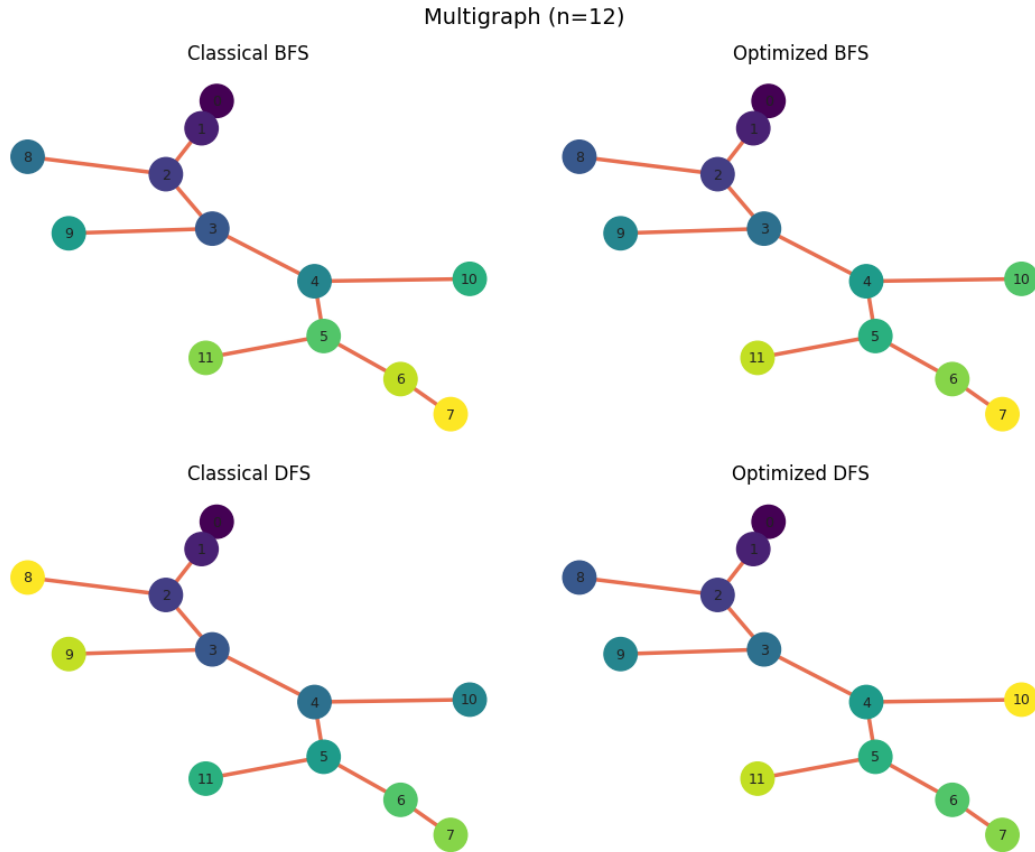


Figure 4: *Multigraph Traversal (n=12)*.

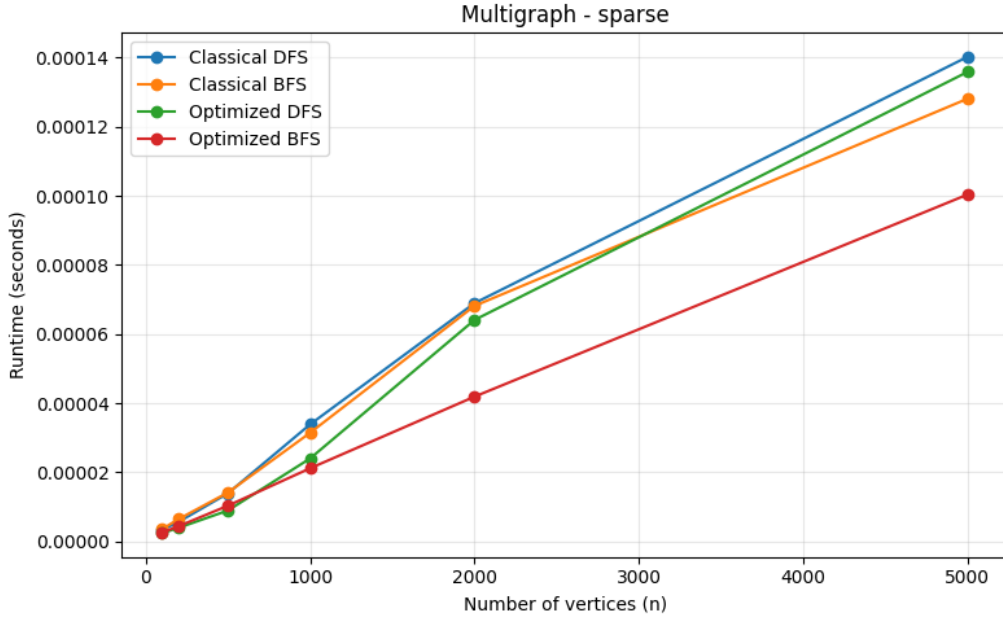


Figure 5: *Multigraph Results – Sparse.*

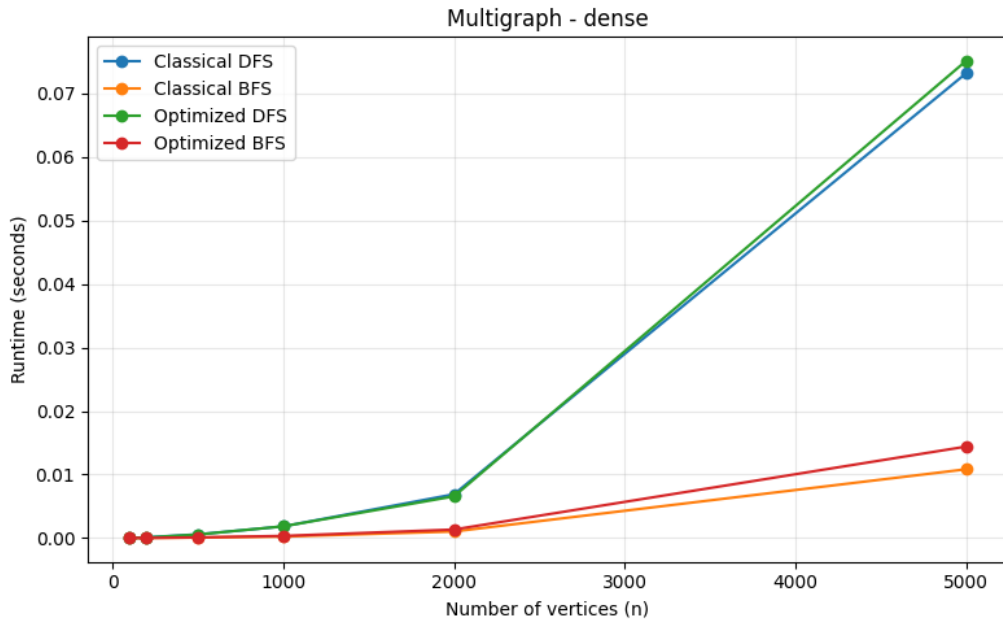


Figure 6: *Multigraph Results – Dense.*

Sparse multigraphs retain near-linear growth. Dense multigraphs climb faster primarily because of the higher target edge count, not because of true parallel edges in the benchmark. The near-overlap with the simple-graph curves confirms that edge deduplication dominates the runtime behavior here.

3. Pseudograph

Multigraph with self-loops in the visualization. In the C++ benchmark generator, self-loops are avoided and edges are deduplicated, so the timed structure again reduces to a simple random graph with the chosen density.

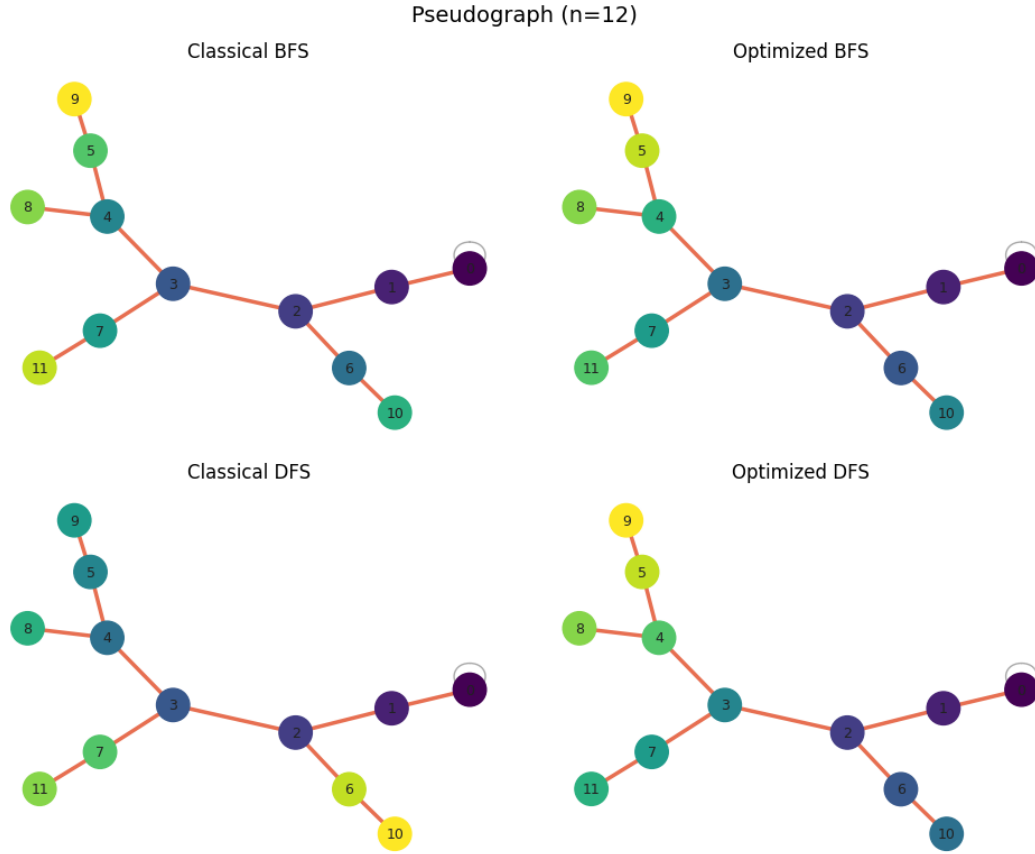


Figure 7: *Pseudograph Traversal (n=12)*.

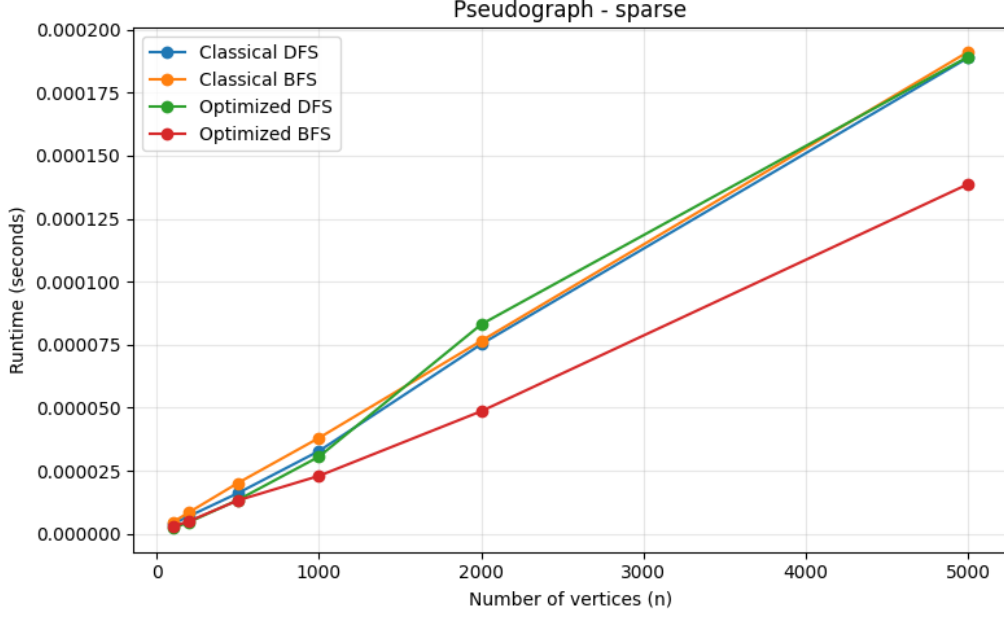


Figure 8: *Pseudograph Results – Sparse.*

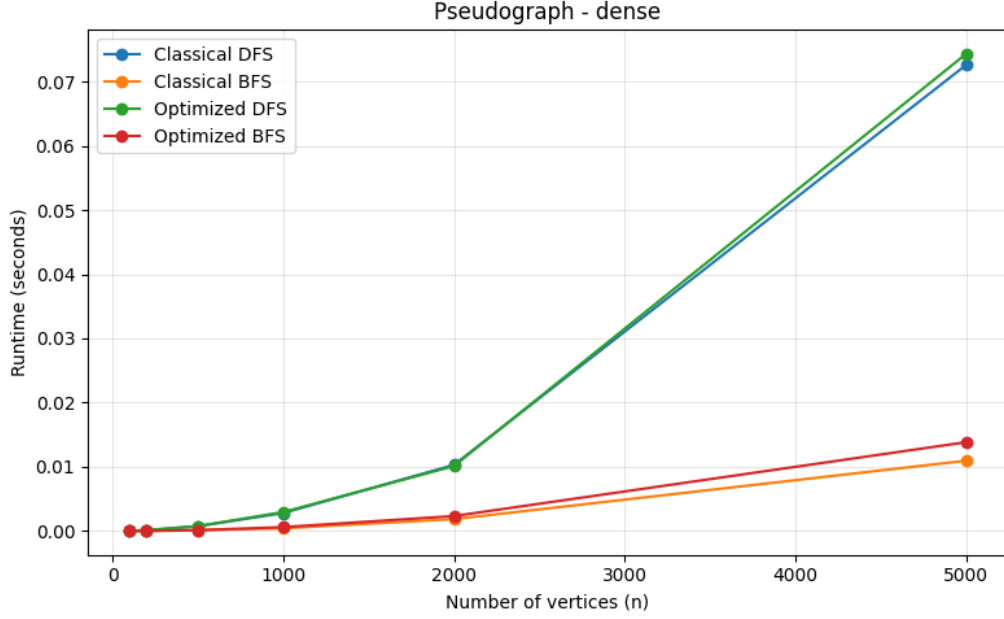


Figure 9: *Pseudograph Results – Dense.*

Runtime trends mirror multigraph behavior in both regimes because self-loops do not appear in the adjacency lists used for timing. The sparse case is close to linear, while dense settings grow faster due to the larger edge set.

4. Directed Graph

Edges are ordered. The generator samples directed edges uniformly and removes duplicates, so the adjacency lists represent unique arcs only. Directionality reduces reciprocity,

which can narrow the frontier in sparse cases.

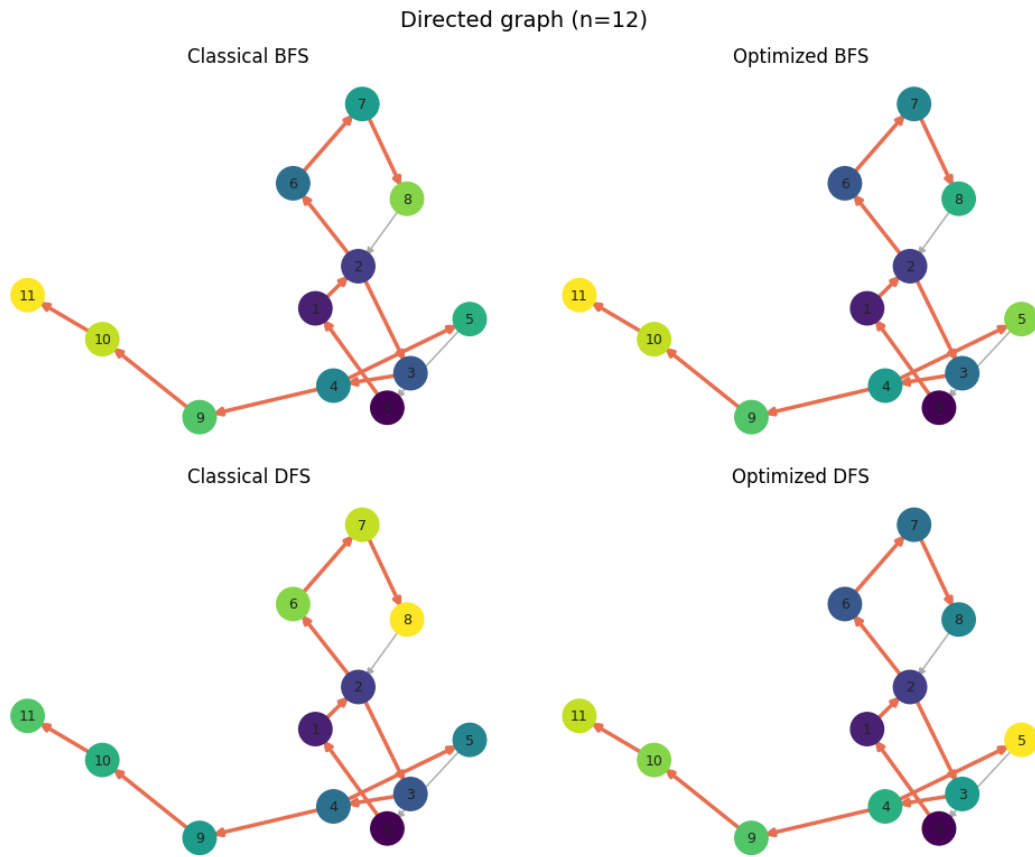


Figure 10: *Directed Graph Traversal (n=12).*

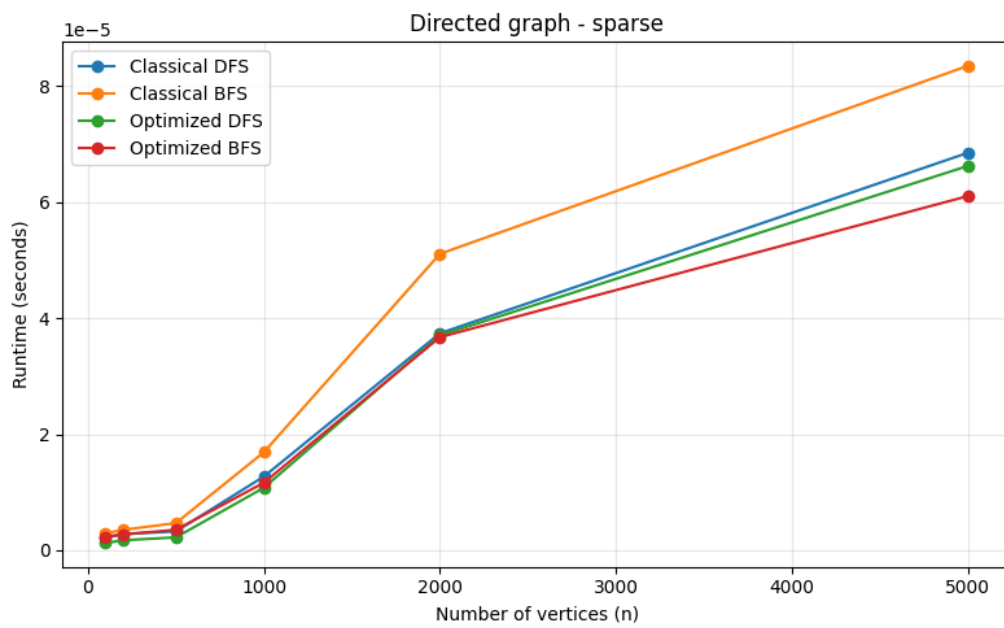


Figure 11: *Directed Graph Results – Sparse.*

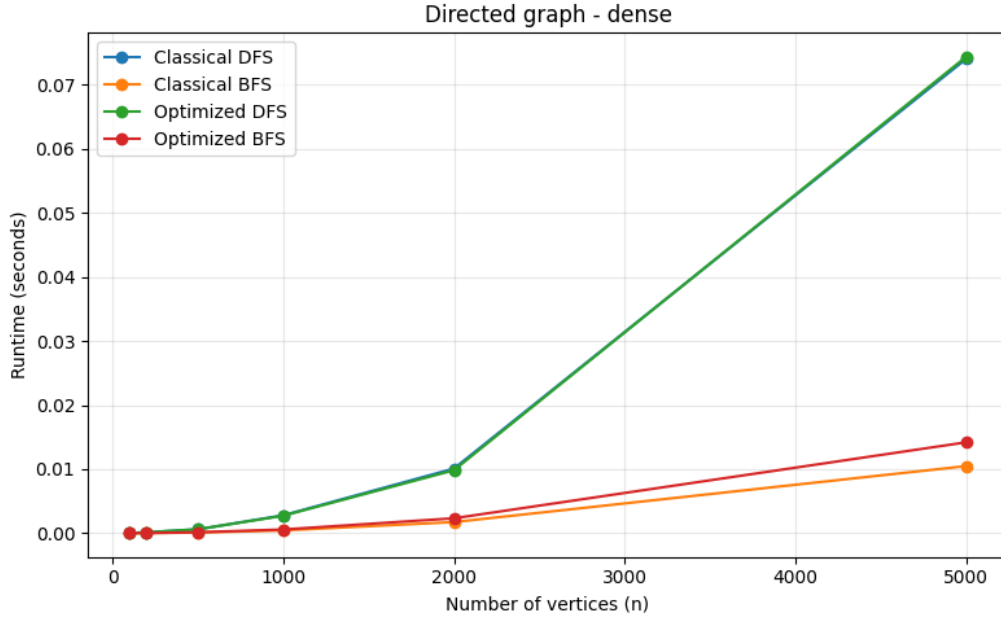


Figure 12: *Directed Graph Results – Dense.*

Sparse directed graphs show small constants due to lower out-degree and fewer outgoing arcs per node. Dense directed graphs increase sharply as out-degree grows and adjacency scans dominate. BFS and DFS remain close because both are bounded by scanning the same adjacency lists.

5. Directed Multigraph

Directed edges with parallel arcs in the visualization. In the benchmark code, parallel arcs are deduplicated, so the timed graph is effectively a directed simple graph with the target density.

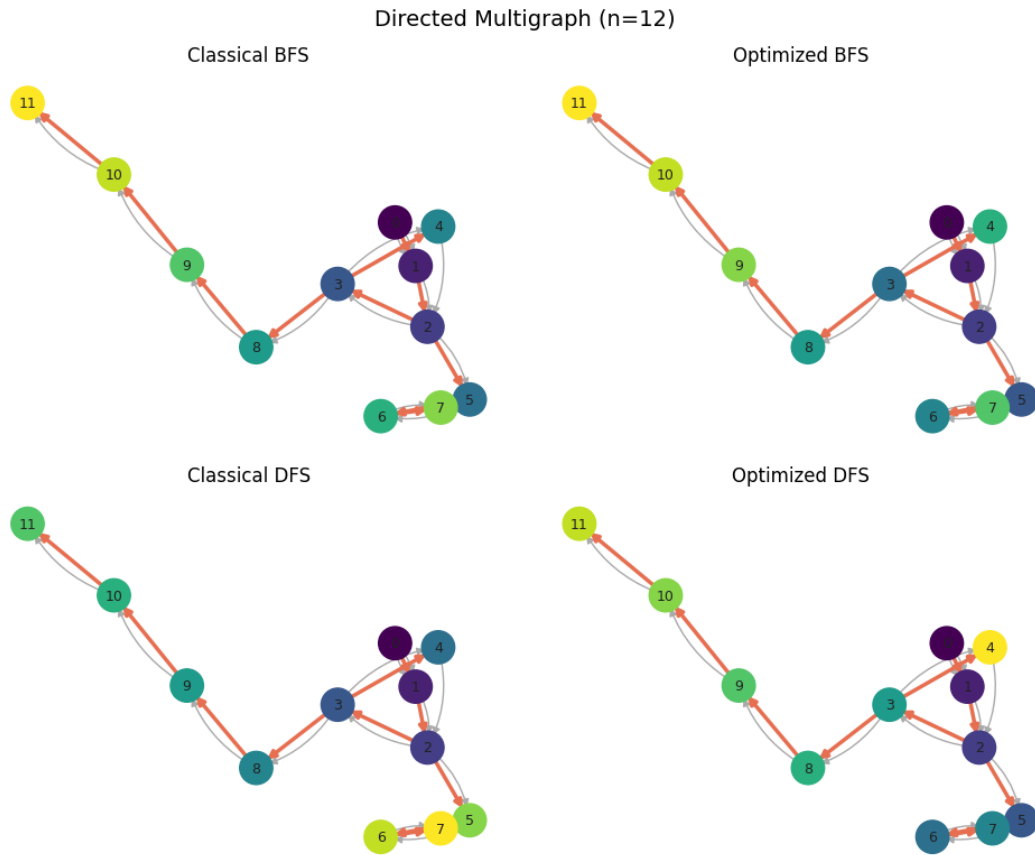


Figure 13: *Directed Multigraph Traversal (n=12).*

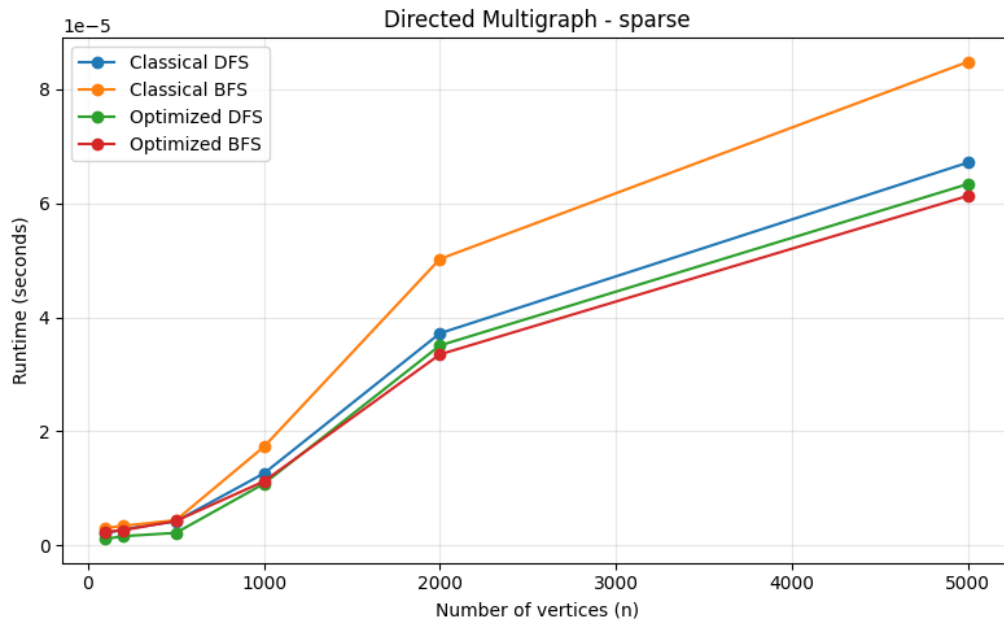


Figure 14: *Directed Multigraph Results – Sparse.*

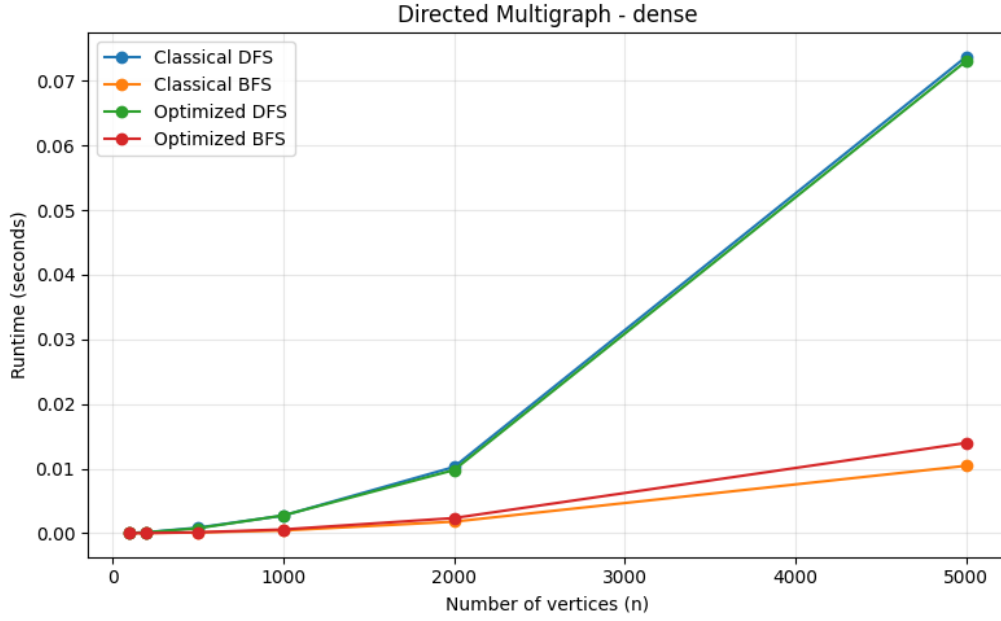


Figure 15: *Directed Multigraph Results – Dense.*

The dense case rises quickly because each arc is still scanned even if it targets a previously visited node. Sparse results are close to directed graphs because the generator removes duplicates and keeps unique arcs only.

6. Complete Graph

Every pair of nodes is connected in the dense setting, which yields the maximum edge count $E = O(n^2)$. In the sparse setting, the generator samples a random subset of edges, so the graph is not fully complete.

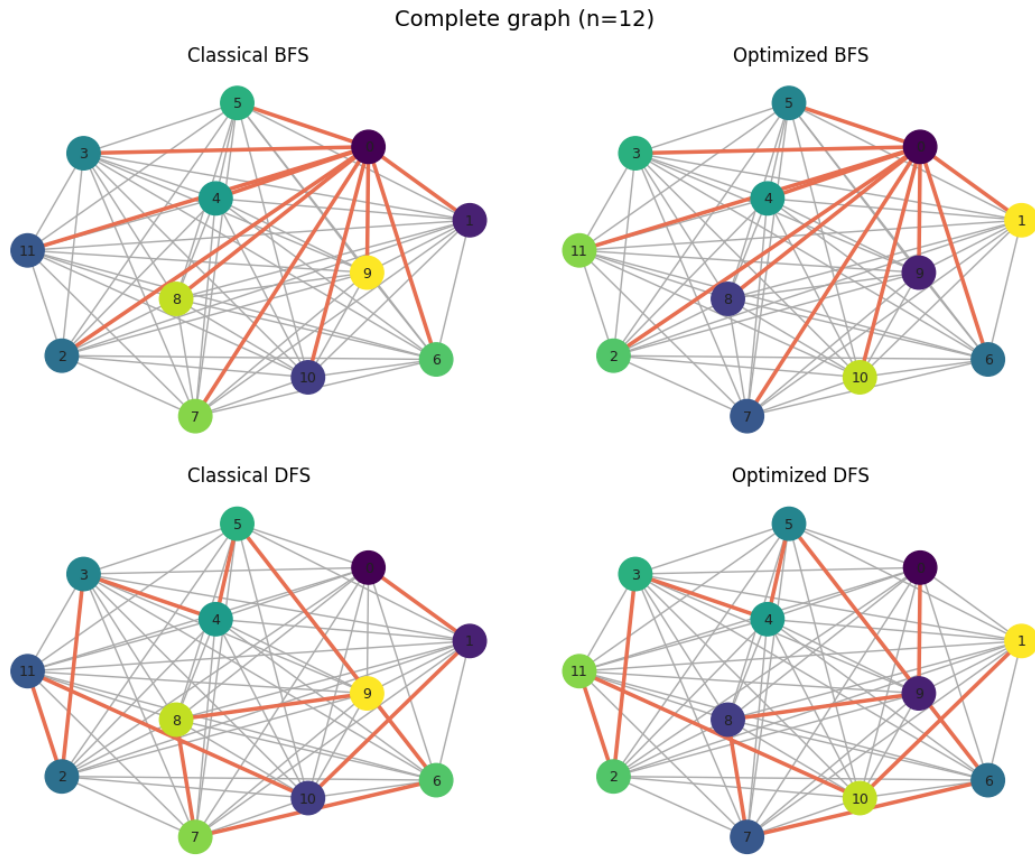


Figure 16: *Complete Graph Traversal ($n=12$).*

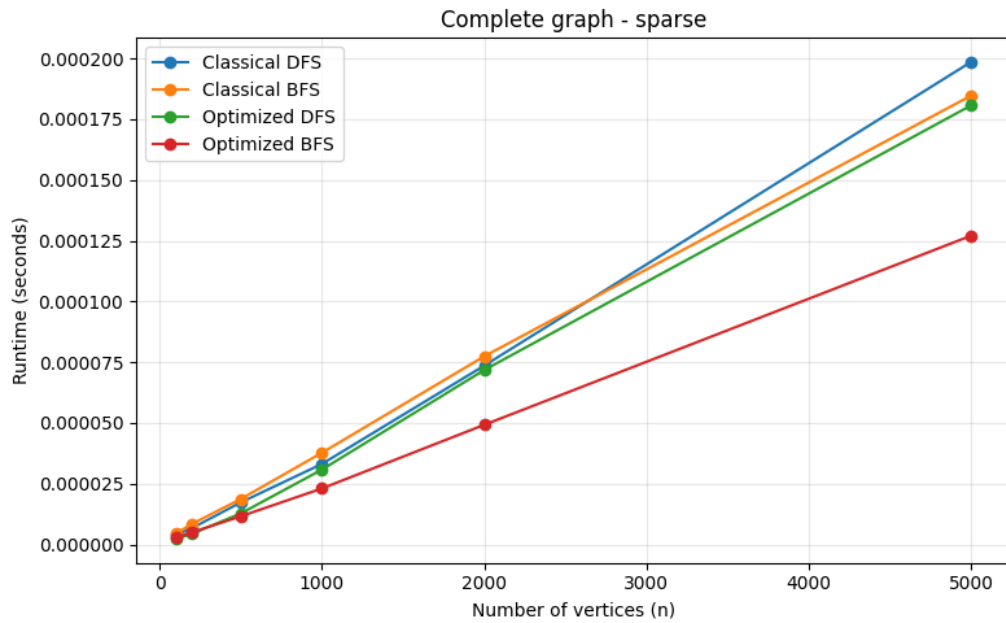


Figure 17: *Complete Graph Results – Sparse.*

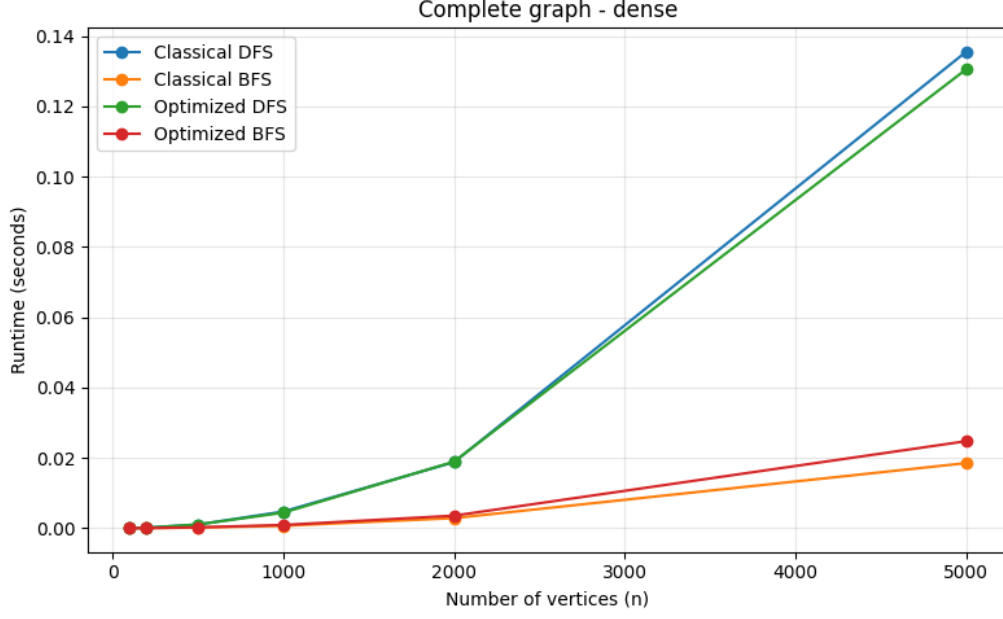


Figure 18: *Complete Graph Results – Dense.*

Both regimes show strong super-linear growth once the edge set becomes large. The dense complete case is the most expensive because every node has degree $n-1$, so scanning dominates and algorithm choice matters less. The sparse case sits closer to linear but still above simple graphs due to higher average degree.

7. Wheels

Wheel graphs combine a cycle with a central hub. The base structure has small diameter, so BFS frontiers expand quickly, and DFS can reach the hub early and then follow the rim. For dense mode, extra edges are added on top of the wheel.

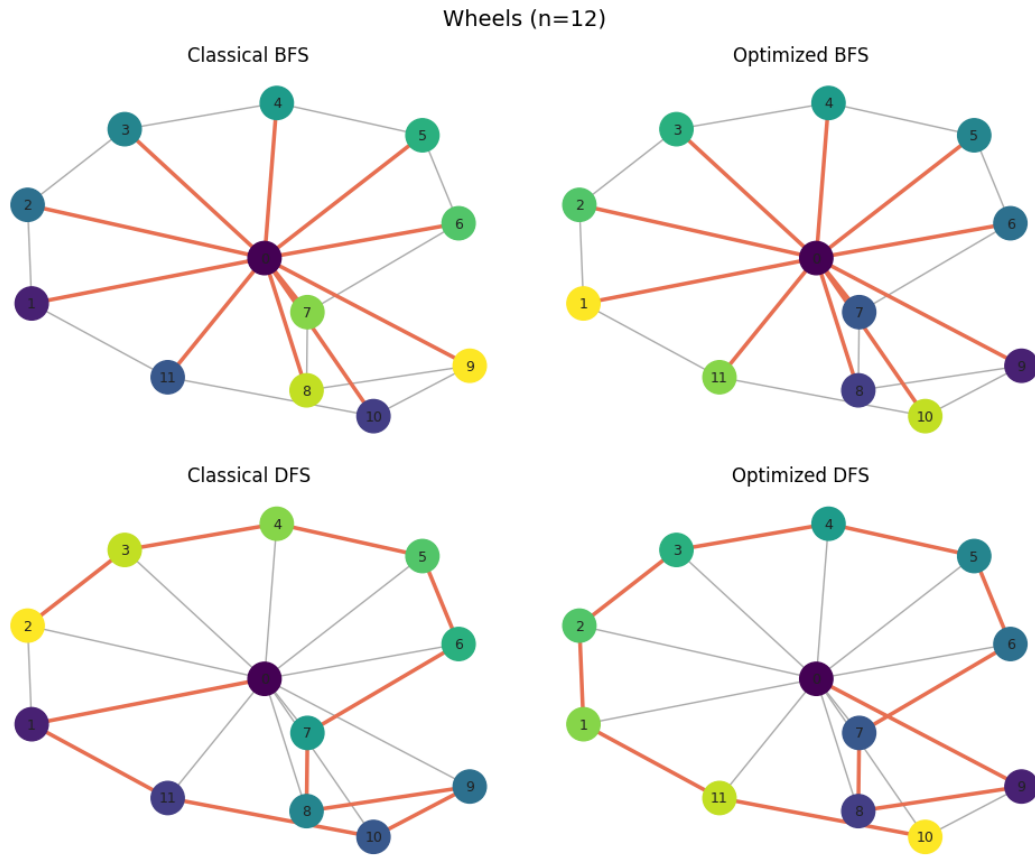


Figure 19: *Wheel Graph Traversal (n=12).*

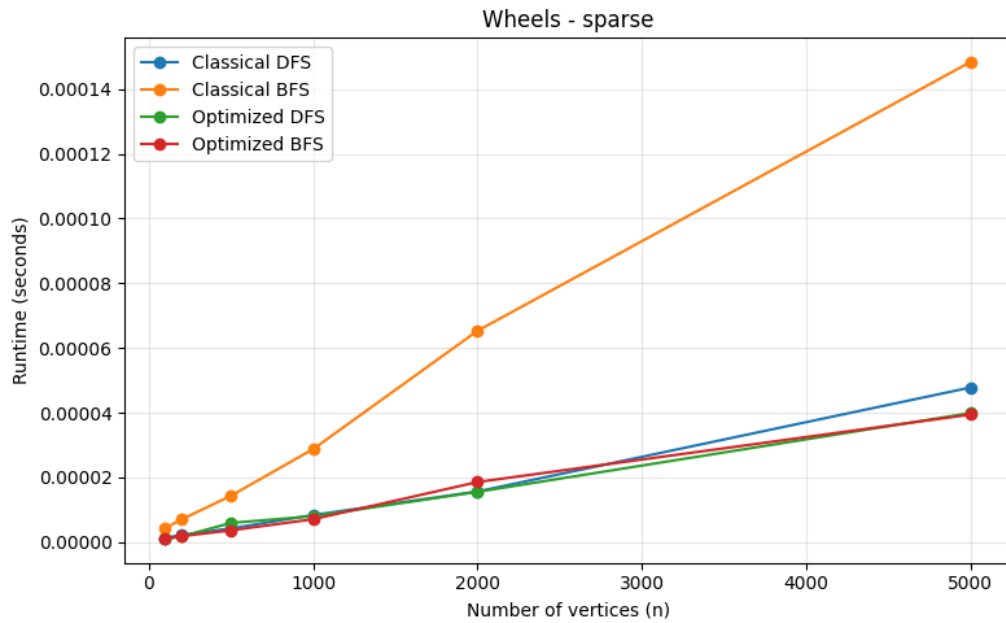


Figure 20: *Wheel Graph Results – Sparse.*

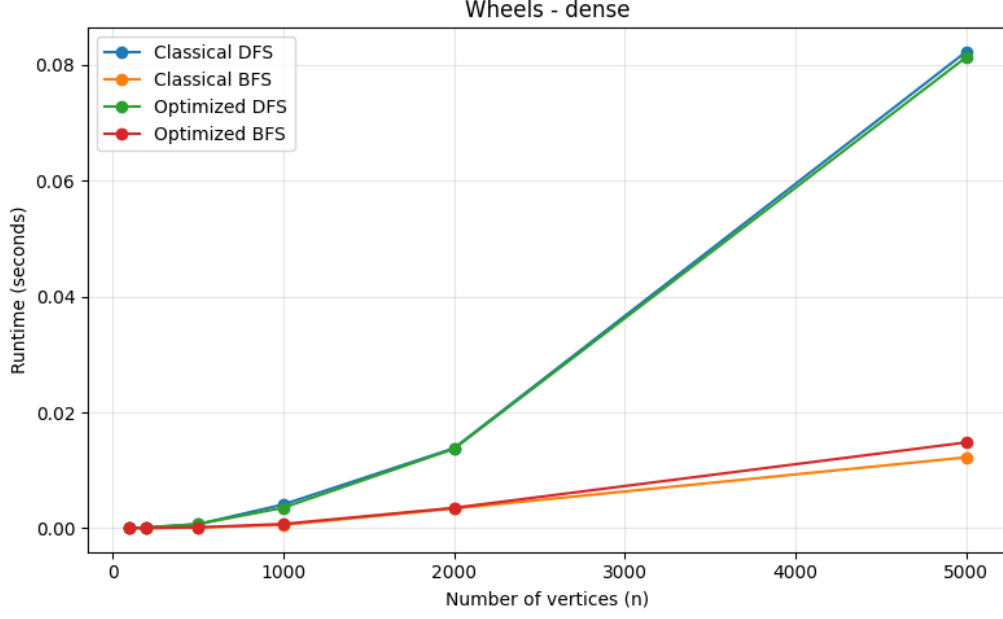


Figure 21: *Wheel Graph Results – Dense.*

In sparse wheels, BFS tends to be competitive because the hub connects to many nodes early and the frontier covers the graph in a few layers. Dense wheels add random edges that reduce the effective diameter further but also increase scan cost, so runtime grows mainly with adjacency size rather than traversal order.

8. Bipartite Graph

Nodes are split into two partitions with edges only between partitions. The generator creates a random bipartite graph with a target edge count, so degree is concentrated across the cut. Dense cases approach a large fraction of the complete bipartite edge set.

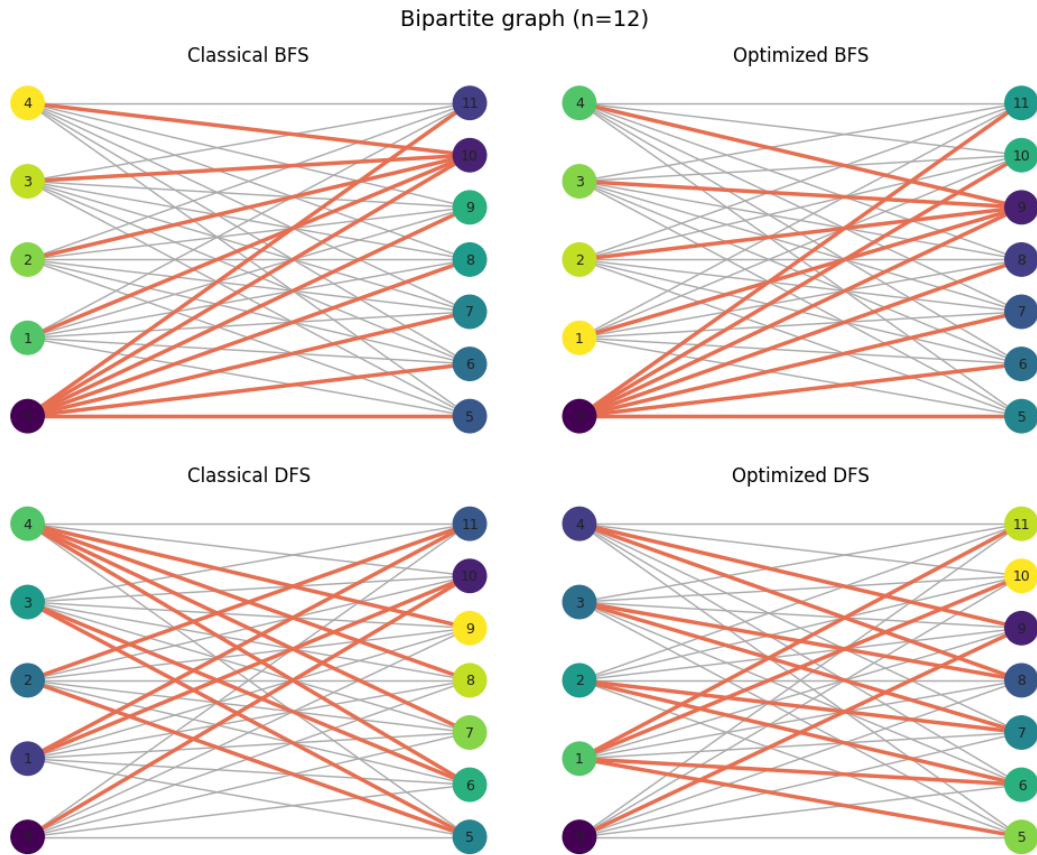


Figure 22: *Bipartite Graph Traversal (n=12).*

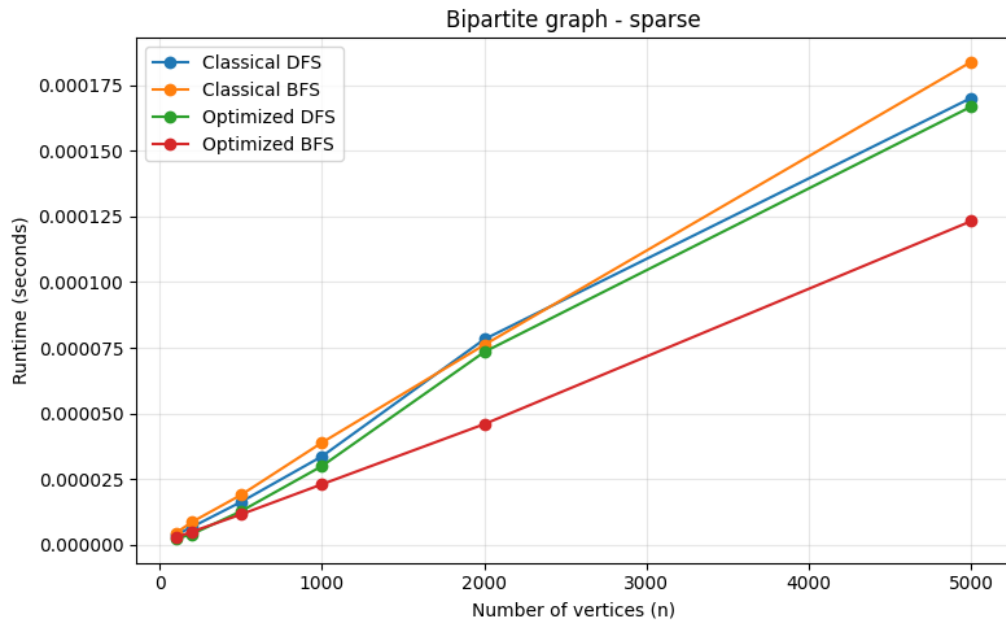


Figure 23: *Bipartite Graph Results – Sparse.*

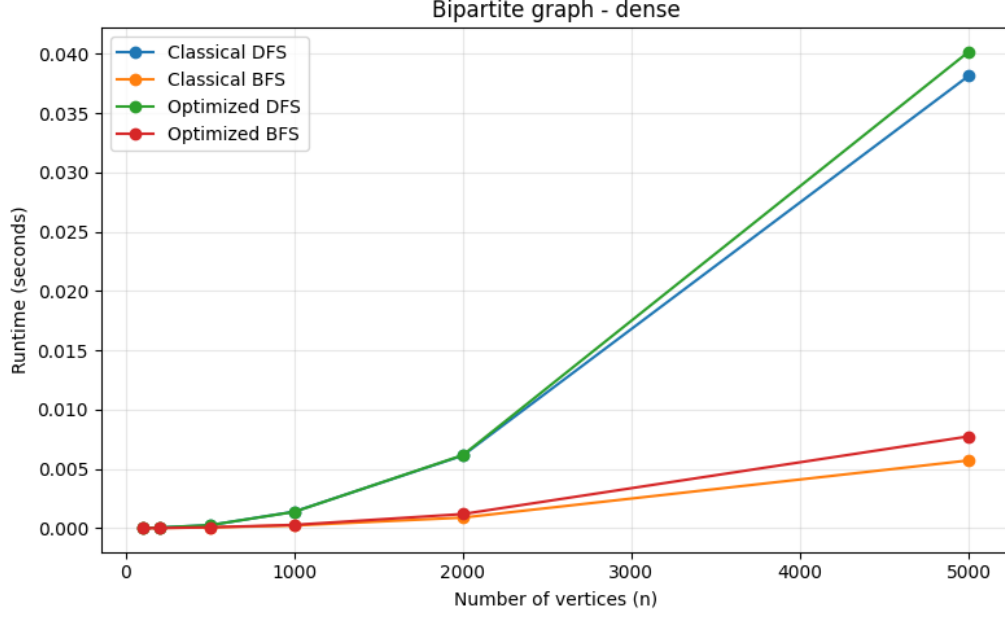


Figure 24: *Bipartite Graph Results – Dense.*

Dense bipartite graphs show steep growth because each side connects to many nodes, causing large adjacency scans. BFS and DFS remain close in runtime, with differences dominated by constant factors and degree-based neighbor ordering.

9. Regular Graph

Each node has the same degree in the base generator; for dense mode the degree is increased toward $n/2$ (while still valid). This reduces topology variance and highlights constant-factor differences in traversal implementations.

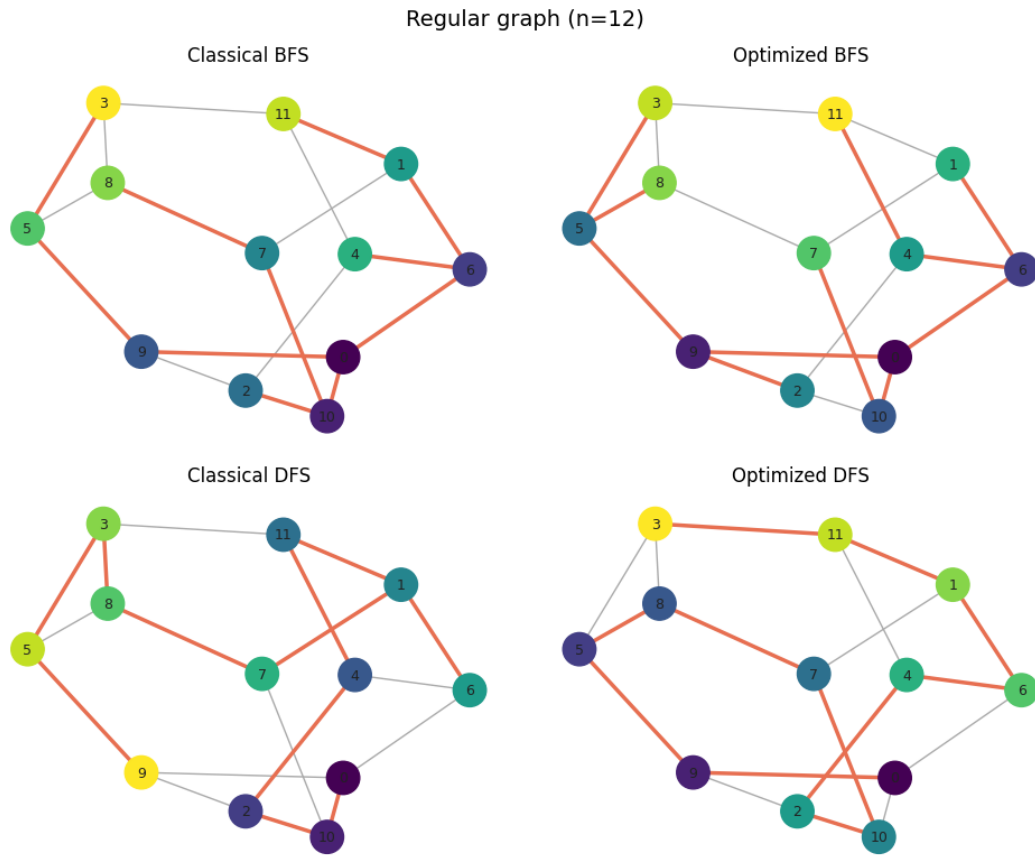


Figure 25: *Regular Graph Traversal (n=12).*

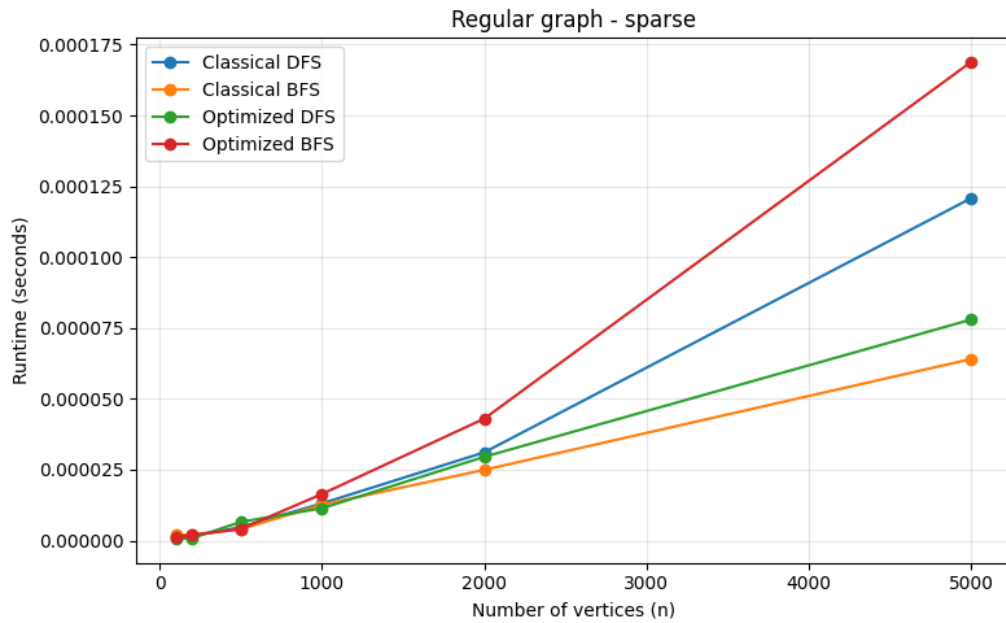


Figure 26: *Regular Graph Results – Sparse.*

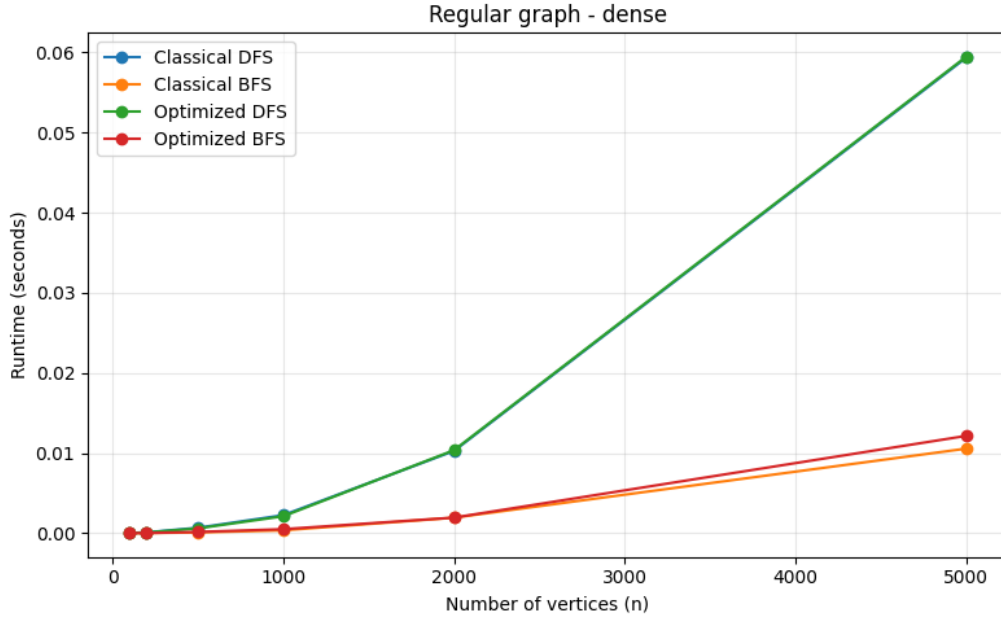


Figure 27: *Regular Graph Results – Dense.*

Because degree is fixed in the sparse regime, the curves are tightly grouped and mostly linear. In the dense regime the degree grows, so adjacency scans increase and the curves rise more steeply, though DFS and BFS remain close.

10. Planar Graph (Grid-Based)

2-D grid-like planar graph. The base grid has bounded degree and a geometric structure that makes BFS expand in layers. For dense mode, random edges are added on top, which can break planarity but preserve the base layout.

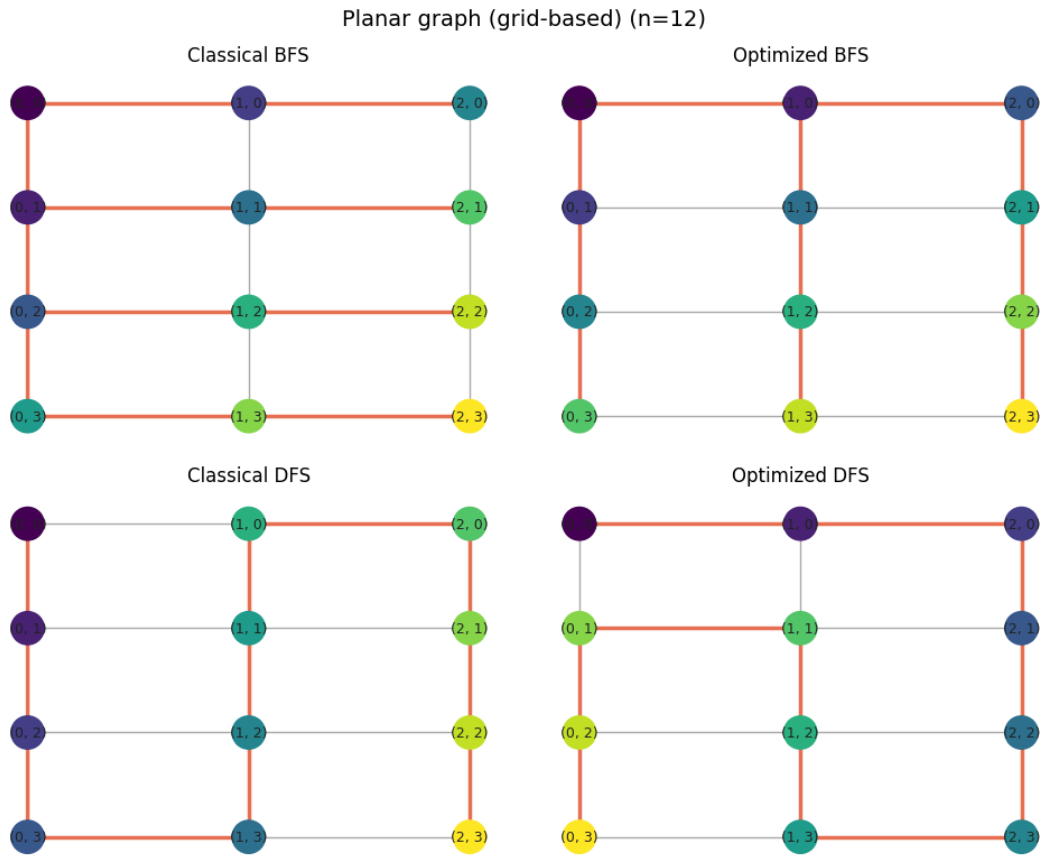


Figure 28: *Planar Graph Traversal ($n=12$).*

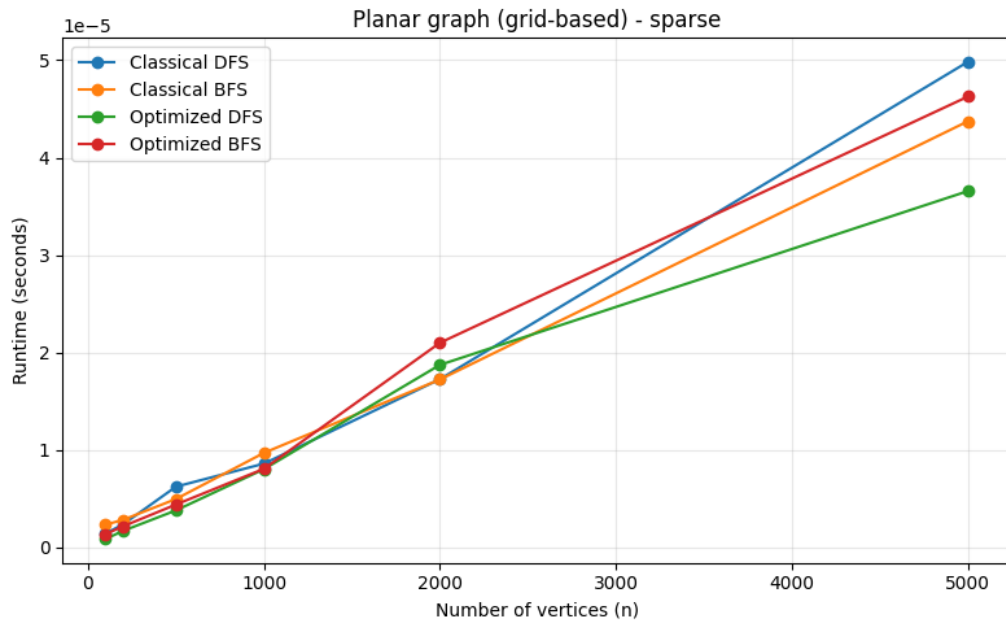


Figure 29: *Planar Graph Results – Sparse.*

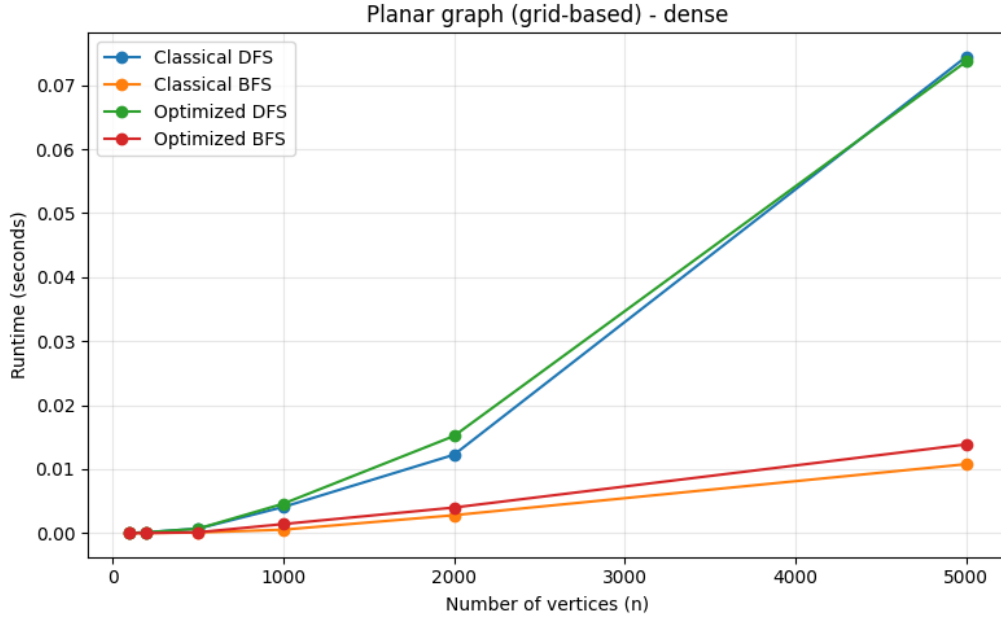


Figure 30: *Planar Graph Results – Dense.*

Grid graphs show close BFS/DFS timings in sparse settings because degree is bounded and $E = O(V)$. Dense variants scale up due to extra edges added on top of the grid, so adjacency scanning dominates.

11. Trees

Connected acyclic graphs. The base generator produces a random tree, so the edge count is $E = V - 1$ and the structure is strictly sparse. Dense mode adds extra edges, so the structure is no longer a tree but still tree-like.

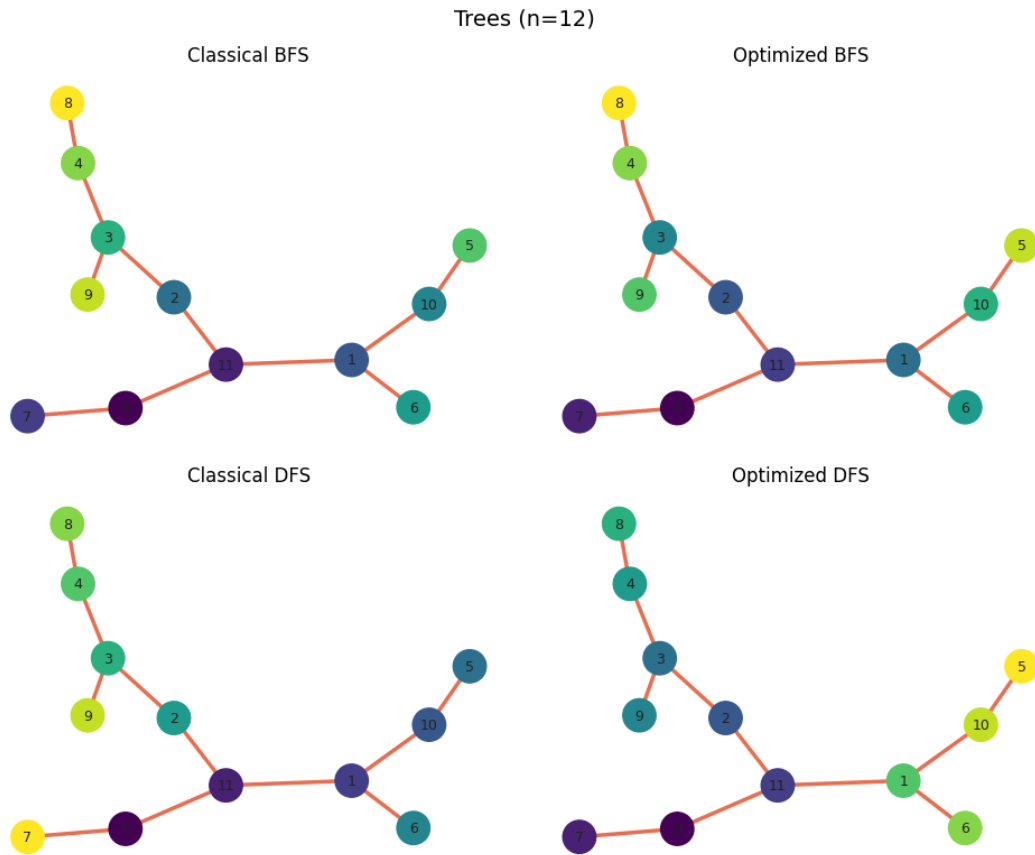


Figure 31: *Tree Traversal (n=12).*

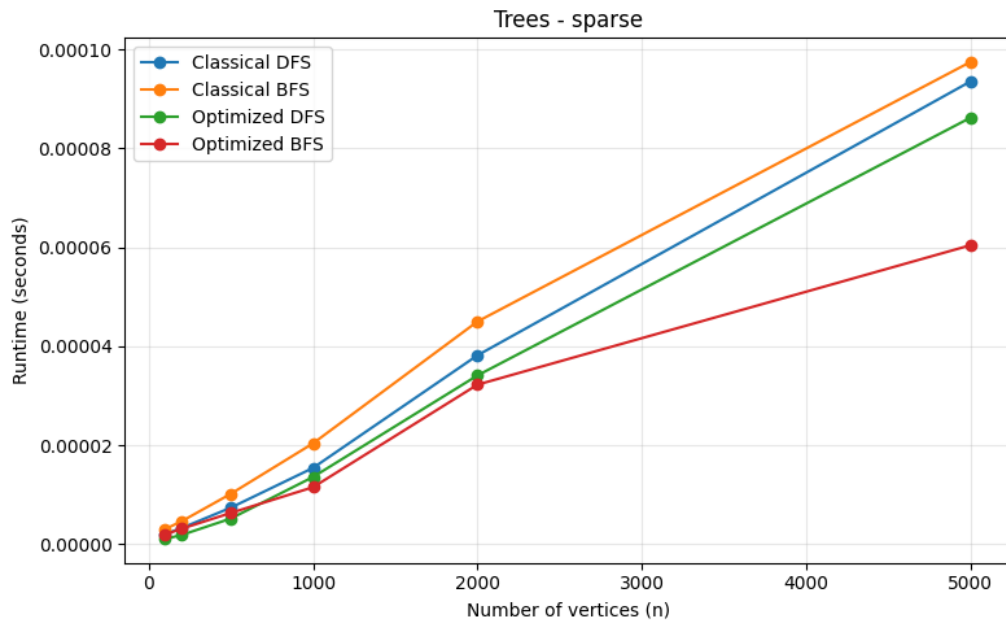


Figure 32: *Tree Results – Sparse.*

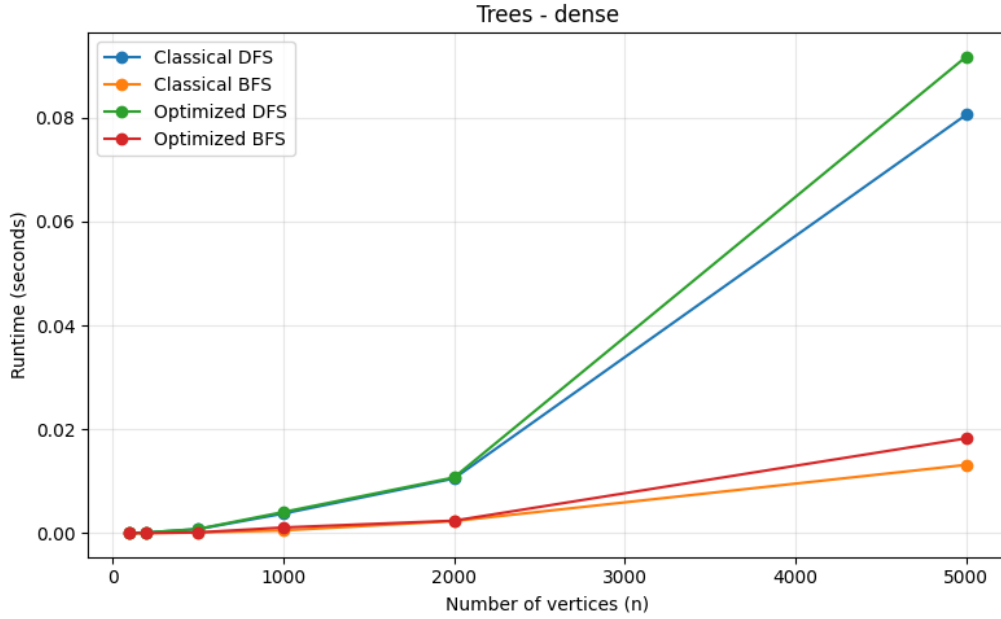


Figure 33: *Tree Results – Dense.*

Trees are close to linear in sparse mode because the edge count grows with V . BFS can hold larger frontiers on wide trees, while DFS follows long branches, but runtime differences are small. In dense mode, added edges increase scan costs and the curves rise more quickly.

12. Cyclic Graph

Cycle graph C_n with degree 2 at each node. The base structure is a single loop, so both traversals advance steadily around the ring. Dense mode adds extra edges that reduce the effective diameter but increase adjacency work.

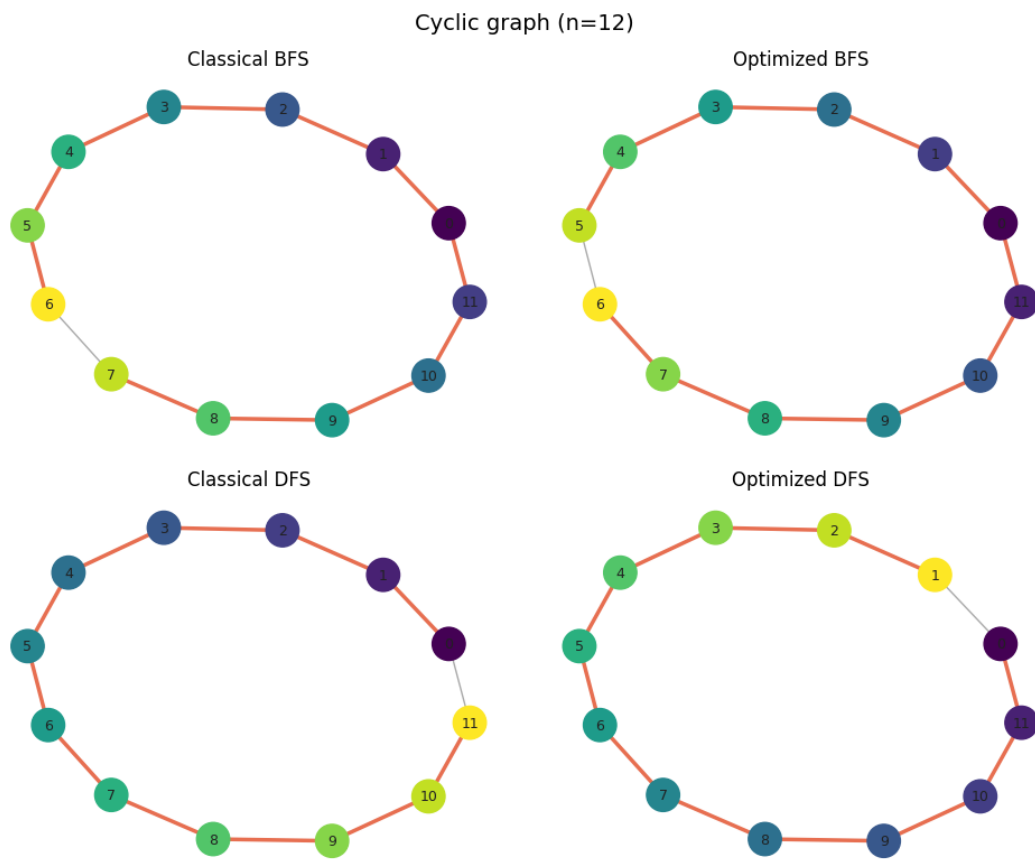


Figure 34: *Cyclic Graph Traversal (n=12).*

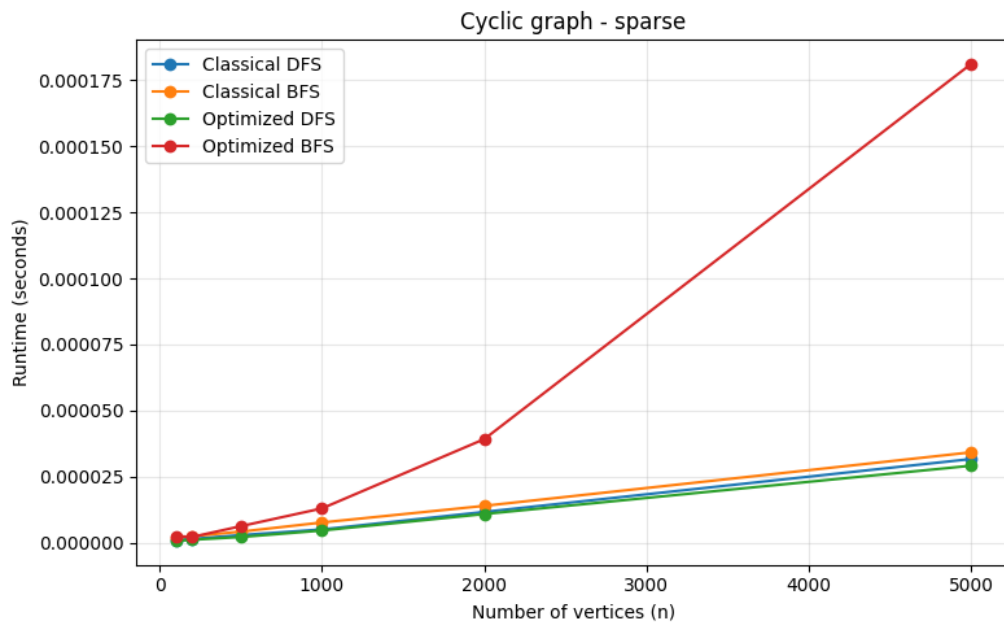


Figure 35: *Cyclic Graph Results – Sparse.*

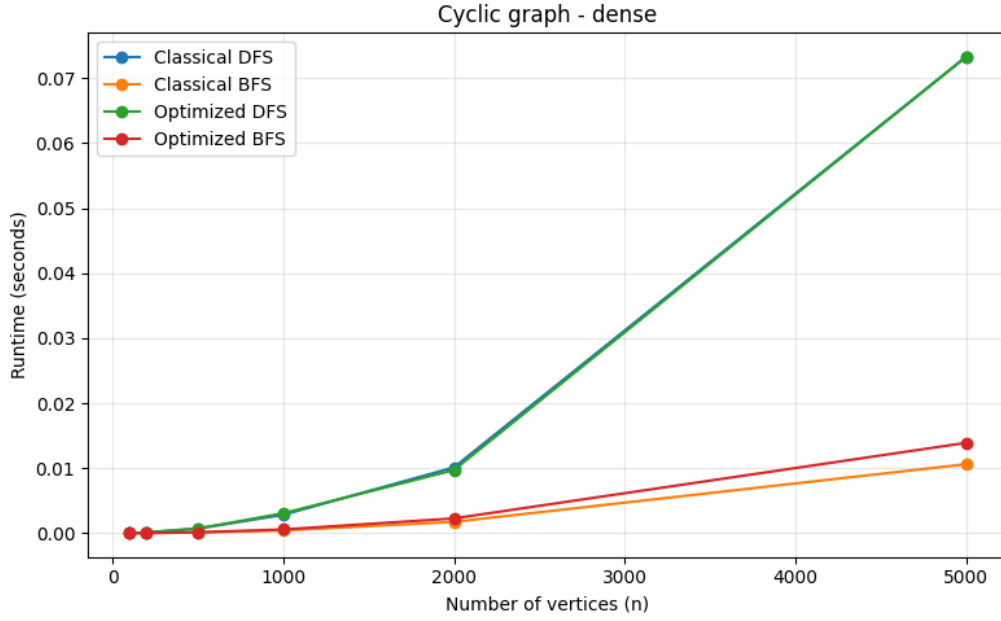


Figure 36: *Cyclic Graph Results – Dense.*

Both DFS and BFS are nearly linear on cycles in sparse mode because the frontier is narrow and each node has constant degree. In dense mode, additional edges increase the scan cost, and the curves rise more sharply while remaining close.

13. Weighted Graph

Undirected graph with integer weights on edges. The benchmarking code sets a weighted flag but uses the same edge generator and adjacency lists, so weights do not influence traversal in the timing runs.

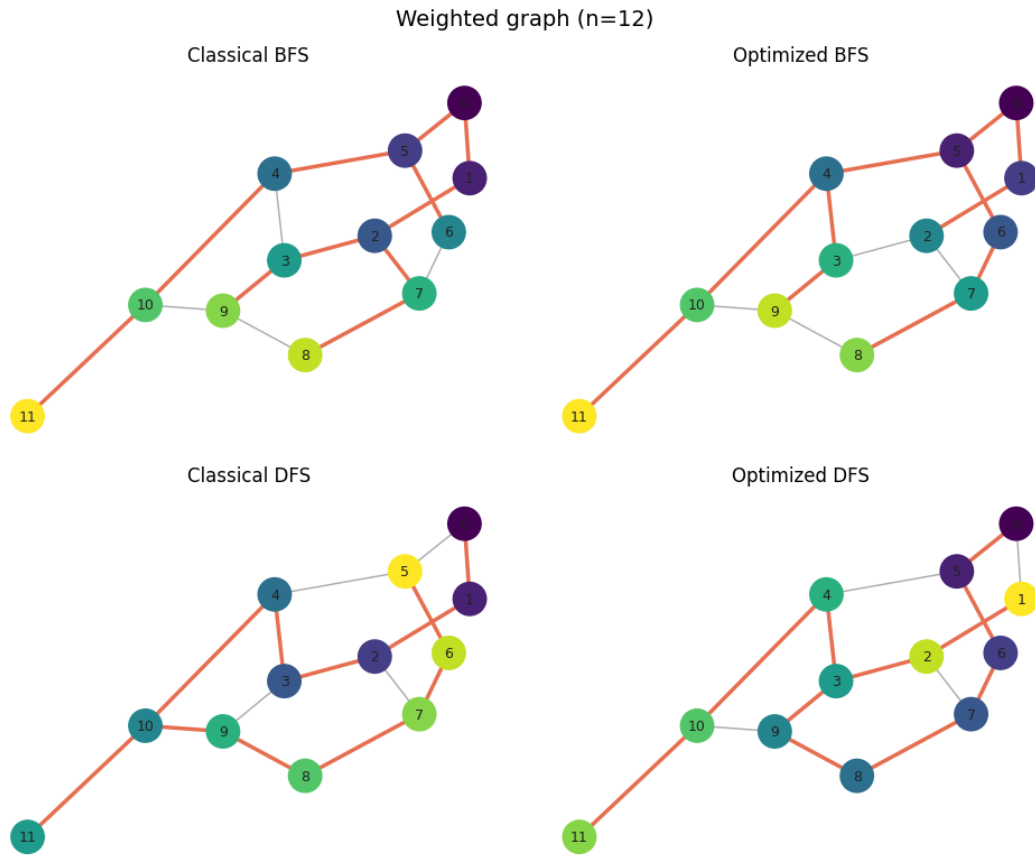


Figure 37: *Weighted Graph Traversal (n=12).*

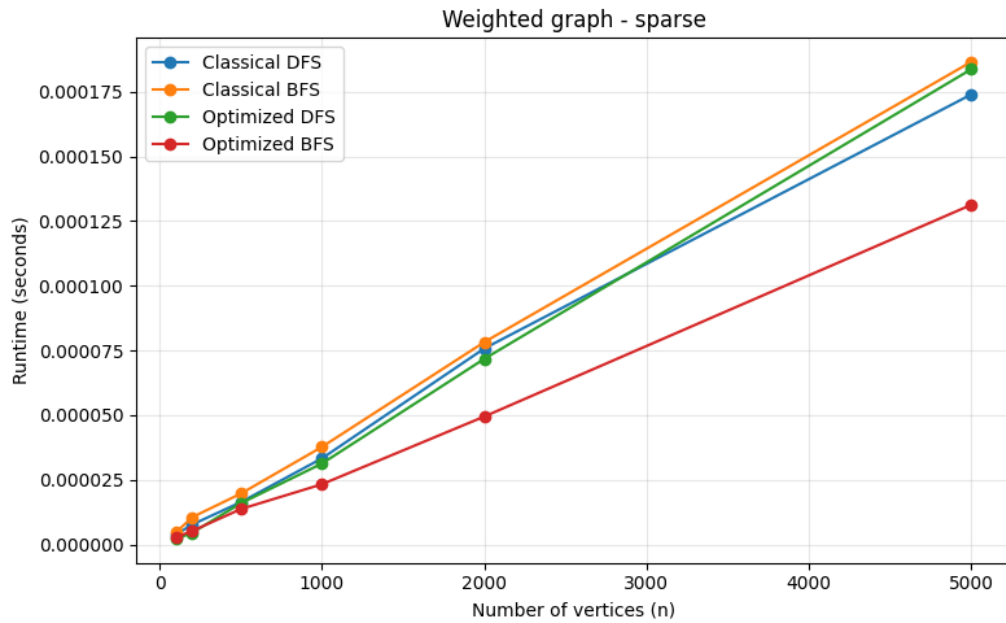


Figure 38: *Weighted Graph Results – Sparse.*

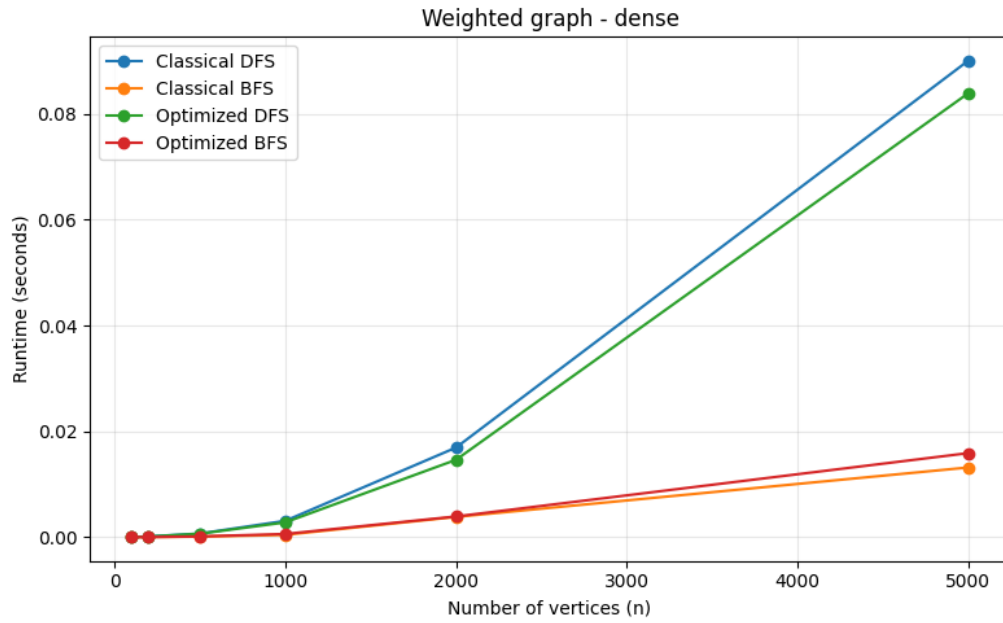


Figure 39: *Weighted Graph Results – Dense.*

Trends mirror the simple-graph case, confirming that weights do not affect traversal cost when they are not used in decision making. Any differences are therefore driven by edge density, not by edge attributes.

CONCLUSION

Through this empirical study, both DFS and BFS were implemented in C++ and evaluated across thirteen representative graph categories, covering sparse, dense, structured, directed, multi-edge, weighted, and regular topologies. The C++ backend was driven from Python, with visualizations and plots generated in the notebook.

Both algorithms share the same asymptotic complexity of $O(V + E)$. The measurements confirm near-linear growth for sparse regimes and sharper growth for dense regimes where adjacency scanning dominates. The optimized variants use CSR storage, buffer reuse, and degree-based neighbor ordering (for DFS), which change traversal order but not asymptotic complexity; differences in runtime therefore reflect constant-factor effects driven by graph structure.

Across the studied graph types, topology has a larger influence on performance than the choice between BFS and DFS. Dense complete and bipartite graphs exhibit the steepest increases due to large edge counts, while trees, cycles, and planar grids remain close to linear scaling. Directed and multigraph variants show how extra arc multiplicity or directionality changes the effective adjacency workload.

From a practical perspective, DFS remains useful for structure-oriented tasks such as cycle detection and backtracking, while BFS is preferable for level-order discovery and shortest-hop paths. Weighted graphs require algorithms such as Dijkstra for weight-aware shortest paths; BFS and DFS provide only unweighted traversal. Overall, the experiments confirm that BFS and DFS are comparably efficient in practice, and the best choice depends primarily on the topology and the traversal objective.

REFERENCES

- [1] **P., I. (2026).** *Analysis of Algorithms*. GitHub Repository. https://github.com/Nichita111/AA_labs
- [2] GeeksforGeeks. (2024). *Breadth First Search or BFS for a Graph*. <https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>
- [3] GeeksforGeeks. (2024). *Depth First Search or DFS for a Graph*. <https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [5] Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.